

CS-222

Database Management System



COMPLEX ENGINEERING PROBLEM

TRAFFIC VIOLATION AND FINE MANAGEMENT SYSTEM

By

SYED ARSHMAN ALI SHAH (CS-24025)

MAIR NASIR (CS-24026)

SIRAJ UL HAQ (CS-24034)

Submitted to

DR.HAMEEZA

Abstract

This report details the design and implementation of the Traffic Violation and Fine Management System (TVFMS), a comprehensive database solution developed to digitize and automate traffic enforcement workflows for metropolitan authorities. The project encompasses a complete database development lifecycle, beginning with real-world requirements gathering to formulate a robust Entity-Relationship (ER) model. The conceptual design was subsequently mapped into relational tables and rigorously normalized up to the Third Normal Form (3NF) to ensure data integrity and eliminate redundancies. Physical implementation was achieved using precise SQL Data Definition Language (DDL) statements, incorporating primary constraints, foreign key cascades, and optimized indices for efficient querying.

The system's architecture features a secure, role-based web interface built with HTML, CSS, and JavaScript, powered by a Python (Flask) back-end. Key operational capabilities include automated fine calculation with dynamic repeat-offender algorithms, digital PDF challan generation, secure payment processing, and comprehensive administrative analytics. Ultimately, the TVFMS provides a highly scalable, centralized platform that enhances operational efficiency, transparency, and data accuracy in traffic law enforcement.

Table of Contents

Abstract	2
Table Of Contents	3
Requirement Formulation and Analysis	4-7
Entity-Relationship Model (ER-Model)	8
Reduction of ER Model to Tables	9-10
Normalization	11-18
DDL Statements for Creating Tables	19-21
DDL Statements for Indices	22-23
Languages	24
Implementation Design -----	25-28
Authentication and Authorization -----	29-31
Future Expansion -----	32

Requirement Formulation and Analysis

Introduction

The Traffic Violation and Fine Management System (TVFMS) is designed to serve the needs of a metropolitan city traffic authority. The system digitizes and automates the complete workflow of recording traffic violations, calculating fines, generating digital challans, and tracking payments. To build a database that accurately reflects real-world operations, a thorough requirement formulation and analysis process was carried out using multiple information-gathering sources and techniques.

Sources Employed for Gathering Requirements

The following sources were consulted to identify and collect the data requirements for this system:

Organization Website and Public Records

The official websites of traffic authorities (such as the Lahore Traffic Police and Islamabad Traffic Police) were studied to understand how violations are categorized, what fine amounts are charged for different offenses, how challans are structured, and what information is printed on official challan documents. This provided a strong foundation for identifying the key data entities of the system.

Company / Authority Records

Existing paper-based challan books, manual violation registers, and fine receipt documents used by traffic wardens were examined. These records revealed what data fields are currently captured during a violation (vehicle number, owner name, violation type, location, date, officer badge number, fine amount), and what information is printed on payment receipts. Examining these physical records helped ensure the database captures all fields that were previously managed on paper.

Examination of Similar Existing Systems

Digital traffic enforcement systems from other cities (e-Challan systems used in various Pakistani cities and Indian states) were studied online. This helped identify standard features such as automatic late payment penalties, repeat offender surcharges, and PDF challan generation that should be incorporated into the system.

Analysis of Identified Requirements

Based on the information gathered from the above sources, the following analysis was performed to identify the entities, their attributes, and the relationships between them.

Identified User Views

The system has two primary user views:

User View	Role	Key Data Needs
Traffic Officer	Operational user	Vehicle lookup, violation recording, challan generation, payment processing
Administrator	Managerial user	Reports, officer management, violation type configuration, analytics

Identified Data Requirements

Through analysis of all gathered information, the following major data items were identified as necessary for the system:

User and Officer Data: Every person who uses the system must have a login account with a username, password, and role. Officers additionally have a badge number, department, precinct, and hire date. This led to the identification of two related entities — Users and Officers — where each officer is linked to exactly one user account.

Vehicle and Owner Data: Every vehicle involved in a violation must be traceable to its registered owner. The vehicle data includes registration number, make, model, year, color, and type. The owner data includes CNIC (national identity number), full name, address, and contact information. Analysis revealed a one-to-many relationship: one owner can possess multiple vehicles.

Violation Type Data: Traffic authorities maintain a standard list of violation categories, each with a fixed base fine and severity level. This data needs to be stored separately in its own entity so that fine amounts can be updated centrally without modifying individual violation records.

Violation Data: Each incident of a traffic violation needs to be recorded with the specific vehicle involved, the officer who recorded it, the violation type, the date and time, and the exact location. Analysis also identified the need for automatic repeat-offender detection — the system must check how many times the same vehicle has committed the same violation and apply a fine multiplier accordingly.

Challan Data: Every violation automatically generates one digital challan. The challan has a unique system-generated number, an issue date, a due date (15 days from issue), the total fine amount, the

amount paid so far, and a status (Pending, Partial, Paid, or Cancelled). Analysis of authority records confirmed the one-to-one relationship between a violation and its challan.

Payment Data: One challan can receive multiple payments (partial payments). Each payment records the amount paid, payment method (cash, card, online, bank transfer), date, receipt number, and the officer who processed it. Analysis of late payment rules revealed the need to automatically calculate a 10% weekly penalty (capped at 50%) for overdue challans.

Identified Processing Requirements

Requirement	Description
Automatic fine calculation	System calculates fine at time of violation based on base fine and repeat-offender multiplier
Late penalty calculation	System applies 10% per week penalty for overdue challans, maximum 50% surcharge
Challan number generation	System auto-generates unique challan numbers in format TRF-YYYY-NNNNNN
PDF generation	System must produce downloadable, printable PDF challans
Search and lookup	System must support search by vehicle number, owner CNIC, challan number, and date range
Reporting	Admin dashboard must show monthly statistics, top violations, repeat offenders

Identified Security and Integrity Requirements

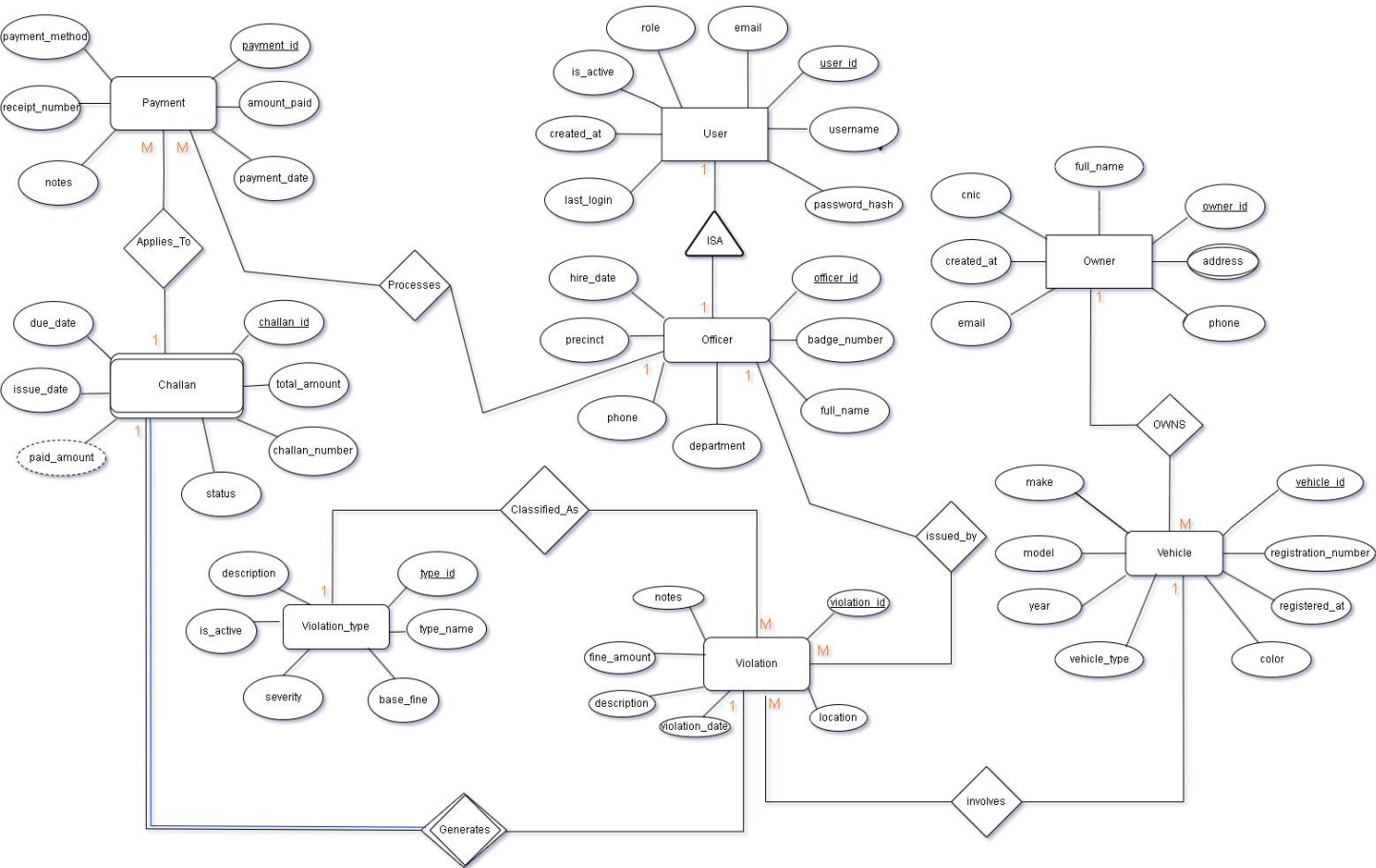
- All passwords must be stored as secure cryptographic hashes (not plain text)
- Officers must only see violations they personally recorded; admins see all records
- Database must enforce referential integrity through foreign key constraints
- All SQL queries must use parameterized inputs to prevent SQL injection attacks
- Sessions must expire after 30 minutes of inactivity

Final List of Identified Entities and Their Attributes

Based on the complete analysis above, the following eight entities were identified for the database:

Entity	Key Attributes	Purpose
Users	user_id, username, password_hash, role, email, is_active	System login credentials and role management
Officers	officer_id, user_id, badge_number, full_name, department, precinct	Traffic officer professional details
Owners	owner_id, cnic, full_name, address, phone, email	Vehicle owner personal information
Vehicles	vehicle_id, owner_id, registration_number, make, model, year, color, type	Registered vehicle details
Violation_Types	type_id, type_name, description, base_fine, severity	Standard violation categories and base fines
Violations	violation_id, vehicle_id, officer_id, type_id, date, location, fine_amount	Individual violation incident records
Challans	challan_id, challan_number, violation_id, due_date, total_amount, paid_amount, status	Digital challan documents
Payments	payment_id, challan_id, amount_paid, method, receipt_number, processed_by	Payment transaction records

Entity-Relationship Model (ER Model):



Reduction of ER Model to Tables:

IDENTIFYING Strong Entity Sets

These are mapped directly to tables. The primary key (PK) is the underlined attribute from your diagram.

User (user_id, username, password_hash, email, role, is_active, created_at, last_login)

Owner (owner_id, cnic, full_name, address, phone, email, created_at)

Violation_Type (type_id, type_name, base_fine, severity, is_active)

Vehicle (vehicle_id, registration_number, make, model, year, color, vehicle_type, registered_at)

ISA (Specialization) Relationships

Since an **Officer** is a specific type of **User** (ISA relationship), we use the "Map to Separate Tables" approach to maintain clean normalization.

Officer (officer_id, *user_id*, badge_number, full_name, department, phone, precinct, hire_date)

Weak Entity Sets & Derived 1:1 Relationships

A **Challan** is generated by a **Violation**. In your logic, every Violation leads to exactly one Challan, effectively making it a dependent entity.

Violation (violation_id, *vehicle_id*, *officer_id*, *type_id*, fine_amount, violation_date, location, description, notes)

Challan (challan_id, *violation_id*, challan_number, issue_date, due_date, total_amount, status)

Relationship Sets (1:M Mapping)

For **1:M** (One-to-Many) relationships like "OWNS" or "Applies_To," the primary key of the "1" side is placed as a Foreign Key (FK) in the "M" side.

Vehicle (vehicle_id, ..., owner_id)

Payment (payment_id, challan_id, processed_by_officer_id, amount_paid, payment_date, payment_method, receipt_number, notes)

Relationship Sets

OWNS (vehicle_id, owner_id)

INVOLVES (violation_id, vehicle_id)

ISSUED_BY (violation_id, officer_id)

CLASSIFIED_AS (violation_id, type_id)

PROCESSES (payment_id, officer_id)

GENERATES (challan_id, violation_id)

APPLIES_TO (payment_id, challan_id)

the underlined attributes represent how these relationships are mapped into your tables. For 1:M (One-to-Many) relationships, the Primary Key of the "Many" side is used to uniquely identify the record, while the Primary Key of the "One" side acts as the Foreign Key.

Multi-valued & Derived Attributes

Multi-valued Attributes: It uses simple attributes for phone and email. If we decide to allow multiple phone numbers per owner, you would derive.

Owner_Phones (owner_id, phone_number)

Derived Attributes: paid_amount in the Challan entity is **not** stored as a physical column in 3NF; it is calculated by summing amount_paid from the Payment table for a specific challan_id.

Normalization Explanation

FOR USER TABLE:

1NF — First Normal Form

Establishes a Primary Key (user_id) to uniquely identify each row in the database and putting values in each column.

User_Id PK	username	Password_hash	role	email	Is_active	Created_at	Last_login
1	Ramish_admin	\$2y\$10\$abc...	admin	ramish@khi.pk	1	2026-04-10 09:00	2026-04-11 16:10
2	Arshman_off	\$2y\$10\$xyz...	officer	arshman@khi.pk	1	2026-04-10 11:30	NULL

2NF — Second Normal Form

Since the Primary Key (user_id) consists of only one column, all non-key attributes like role and is_active necessarily depend on the entire key.

3NF — Third Normal Form

Every non-key attribute (e.g., email, password_hash, last_login) depends directly on the user_id and does not depend on any other non-key column.

FOR OWNER TABLE:

1NF — First Normal Form

We establish owner_id as the Primary Key (PK) to uniquely identify each row. To resolve the multiple phone numbers while matching your schema, the business logic restricts the entry to one primary contact number per owner

Owner_id (PK)	CNIC	Full_name	address	phone	email
1	42201-1234567-1	Tariq Khan	Clifton, Karachi	0300-1111111	tariq@khi.com
2	35202-9876543-2	Sarah Raza	DHA Phase 5, Lahore	0300-2222222	sara@lhr.com

2NF — Second Normal Form

- The table must fully satisfy all requirements of 1NF
- We must eliminate partial dependencies. Because the primary key (owner_id) is exactly one column, it is mathematically impossible to have a partial dependency. All data depends on the entire key.
- No structural changes needed from 1NF

3NF — Third Normal Form

Non-key attributes must rely *only* on the primary key. In this table, full_name, address, phone, and email describe the owner directly and do not depend on each other. Therefore, the table is already in 3NF.

FOR OFFICER TABLE:

1NF — First Normal Form

- Every attribute (like badge_number and phone) contains only one value. No multi-valued lists are allowed in a single cell
- The officer_id is established as the unique identifier for every officer in the system.

Officer_id (PK)	User_id (F)	Badge_number	Full_name	department	phone	precinct	Hire_date
1	2	B-774	Arshman Ali	Traffic Police	0333- 1234567	South District	2026-01- 15
2	3	B-890	Siraj Ul Haq	Highway Patrol	0321- 7654321	East District	2026-02- 20

2NF — Second Normal Form

- All non-key attributes must depend on the *entire* primary key. Since officer_id is a single-column primary key, every attribute (like precinct and hire_date) inherently depends on the whole key.
- No structural changes needed

3NF — Third Normal Form

No non-key attribute should depend on another non-key attribute. In this schema, department, precinct, and hire_date all describe the specific officer directly. There is no hidden relationship between non-key columns (e.g., a precinct doesn't force a specific department), so it is already in 3NF.

FOR VEHICLE TABLE:

1NF — First Normal Form

- Every attribute, including color and vehicle_type, contains only one value. There are no repeating groups (e.g., a car cannot have multiple registration numbers in one cell).
- The vehicle_id uniquely identifies each vehicle, ensuring that even two identical white Corollas are tracked as distinct entities.

Vehicle_id (PK)	Owner_id (FK)	Registration_number	Make	Model	Year	color	Vehicle_type	Registered_at
1	101	KHI-789	Toyota	Corolla	2022	White	Sedan	2026-04-10 10:00
2	102	LHR-456	Honda	Civic	2023	Black	Sedan	2026-04-10 11:30

2NF — Second Normal Form

- The table meets all 2NF requirements.
- Since the Primary Key (vehicle_id) is a single column, all non-key attributes (like make, model, and year) depend on the whole key. It is inherently in 2NF.

3NF — Third Normal Form

Non-key attributes depend only on the primary key. In your design, attributes like make and model describe the vehicle itself. Because you only store the owner_id (FK) and not the owner's name or address here, there are no transitive dependencies.

FOR VIOLATION_TYPE TABLE:

1NF — First Normal Form

We establish type_id as the Primary Key (PK) to uniquely identify each violation type. Every attribute contains only one value per cell — there are no repeating groups or multi-valued attributes

Type_id (PK)	Type_name	description	Base_fine	severity	is_active
1	Over speeding	Vehicle speeding exceeded limit	3000.00	moderate	1
2	Drunk Driving	Driving Under Influence	25000.00	critical	1

2NF — Second Normal Form

- The table meets all 2NF requirements.
- Since the Primary Key (type_id) is a single column, it is mathematically impossible to have a partial dependency.
- All non-key attributes (type_name, description, base_fine, severity, is_active) depend on the entire primary key.
- No structural changes needed from 1NF. The table is in Second Normal Form.

3NF — Third Normal Form

- Every non-key attribute depends only on type_id and not on any other non-key column.
- severity does not determine base_fine — two violations can share the same severity but have different fines. No transitive dependencies exist. The table is in Third Normal Form.

FOR VIOLATION TABLE:

1NF — First Normal Form

We establish violation_id as the Primary Key (PK) to uniquely identify each violation record. Every attribute contains exactly one atomic value per cell.

Violation_id(PK)	Vehicle_id(FK)	Officer_id	Type_id(FK)	Violation_date	location	description	Fine_amount	notes
1	1	1	2	2026-04-10 09:30	Shahrah-e-Faisal	Ran red light	5000.00	Repeat offender
2	2	1	1	2026-04-10 11:00	M9-Motorway	Exceeded 120 km/hr	3000.00	NULL

2NF — Second Normal Form

- The table meets all 2NF requirements.
- Although vehicle_id, officer_id, and type_id are present, they are foreign keys only — not part of the Primary Key.
- The Primary Key is solely violation_id (surrogate auto-increment) — a single column PK makes partial dependencies impossible.
- No structural changes needed. The table is in Second Normal Form.

3NF — Third Normal Form

- Every non-key attribute depends only on violation_id and not on any other non-key column
- Only IDs are stored as foreign keys — no names, amounts or details from other tables are repeated here, eliminating transitive dependencies.
- No transitive dependencies exist. The table is in Third Normal Form.

FOR CHALLAN TABLE:

1NF — First Normal Form

- Every cell must contain a single, indivisible value. No column (like status or total_amount) can contain a list or multiple entries for a single record.
- A Primary Key (challan_id) must be defined to ensure every financial record is uniquely identifiable and reachable.

challan_id (PK)	challan_number	Violation_Id	Issue_date	Due_date	Total_ammount	Paid_ammount	status
1	CH-001	501	2026-04-12	2026-04-26	1000.00	0.00	unpaid
2	CH-002	502	2026-04-12	2026-04-26	500.00	500.00	paid

2NF — Second Normal Form

- The table must satisfy all requirements of the First Normal Form.
- All non-key attributes must depend on the *entire* primary key. Since the key is a single attribute (challan_id), all columns automatically fulfill this requirement as they cannot depend on only "half" of the key.

3NF — Third Normal Form

No non-key attribute should depend on another non-key attribute. Every column (issue_date, due_date, status, etc.) relates directly to the challan_id. No column depends on a secondary field (e.g., the due_date is fixed to the record, not determined by the status), meeting the 3NF standard.

FOR PAYMENT TABLE:

1NF — First Normal Form

- Every cell must contain a single value. This ensures that even if a challan is paid in multiple installments, each transaction is recorded as a separate row with its own amount_paid and payment_date
- The payment_id is established as the Primary Key (PK) to uniquely identify every financial transaction in the system.

Payment_id	challan_id	Amount_paid	payment_date	Payment_method	Receipt_number	Processed_by	notes
1	1	500.00	2026-04-12	Cash	REC-9901	2	Partial payment
2	1	500.00	2026-04-13	Online	REC-9905	NULL	Final settlement

2NF — Second Normal Form

Since the Primary Key (payment_id) is a single attribute, every non-key column (like receipt_number or payment_method) depends on the entire key. There are no partial dependencies.

3NF — Third Normal Form

No non-key attribute depends on another non-key attribute. amount_paid, receipt_number, and processed_by all relate directly to the specific payment_id. By using foreign keys for challan_id and processed_by, you have avoided storing redundant data, ensuring the table is in 3NF.

DDL Statements for Creating Tables:

```
--  
--  TABLE 1: Users  
--  
CREATE TABLE IF NOT EXISTS Users (  
    user_id      INT          AUTO_INCREMENT PRIMARY KEY,  
    username     VARCHAR(50)  NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    role         ENUM('admin','officer') NOT NULL DEFAULT 'officer',  
    email        VARCHAR(100) NOT NULL UNIQUE,  
    is_active    TINYINT(1)   NOT NULL DEFAULT 1,  
    created_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    last_login   DATETIME     NULL,  
    CONSTRAINT chk_username_len CHECK (CHAR_LENGTH(username) >= 3)  
) ENGINE=InnoDB;
```

```
--  
--  TABLE 2: Officers  
--  
CREATE TABLE IF NOT EXISTS Officers (  
    officer_id   INT          AUTO_INCREMENT PRIMARY KEY,  
    user_id      INT          NOT NULL UNIQUE,  
    badge_number VARCHAR(20)  NOT NULL UNIQUE,  
    full_name    VARCHAR(100) NOT NULL,  
    department   VARCHAR(100) NULL,  
    phone        VARCHAR(20)  NULL,  
    precinct     VARCHAR(50)  NULL,  
    hire_date    DATE         NULL,  
    CONSTRAINT fk_officer_user  
        FOREIGN KEY (user_id) REFERENCES Users(user_id)  
        ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

-- TABLE 3: Owners

```
CREATE TABLE IF NOT EXISTS Owners (  
    owner_id    INT          AUTO_INCREMENT PRIMARY KEY,  
    cnic        VARCHAR(15)  NOT NULL UNIQUE,  
    full_name   VARCHAR(100) NOT NULL,  
    address     TEXT         NULL,  
    phone       VARCHAR(20)  NULL,  
    email       VARCHAR(100) NULL,  
    created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT chk_cnic_len CHECK (CHAR_LENGTH(cnic) >= 13)  
) ENGINE=InnoDB;
```

-- TABLE 4: Vehicles

```
CREATE TABLE IF NOT EXISTS Vehicles (  
    vehicle_id    INT          AUTO_INCREMENT PRIMARY KEY,  
    owner_id      INT          NOT NULL,  
    registration_number VARCHAR(20) NOT NULL UNIQUE,  
    make          VARCHAR(50)  NULL,  
    model         VARCHAR(50)  NULL,  
    year          YEAR         NULL,  
    color         VARCHAR(30)  NULL,  
    vehicle_type  ENUM('car', 'motorcycle', 'truck', 'bus', 'van', 'other') NOT NULL DEFAULT 'car',  
    registered_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_vehicle_owner  
        FOREIGN KEY (owner_id) REFERENCES Owners(owner_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

-- TABLE 5: Violation_Types

```
CREATE TABLE IF NOT EXISTS Violation_Types (  
    type_id    INT          AUTO_INCREMENT PRIMARY KEY,  
    type_name  VARCHAR(100) NOT NULL UNIQUE,  
    description TEXT        NULL,  
    base_fine  DECIMAL(10,2) NOT NULL,  
    severity   ENUM('minor', 'moderate', 'major', 'critical') NOT NULL DEFAULT 'minor',  
    is_active  TINYINT(1)   NOT NULL DEFAULT 1,  
    CONSTRAINT chk_base_fine CHECK (base_fine > 0)  
) ENGINE=InnoDB;
```

```
--  
-- TABLE 6: Violations  
--
```

```
CREATE TABLE IF NOT EXISTS Violations (  
    violation_id INT AUTO_INCREMENT PRIMARY KEY,  
    vehicle_id INT NOT NULL,  
    officer_id INT NOT NULL,  
    type_id INT NOT NULL,  
    violation_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    location VARCHAR(200) NOT NULL,  
    description TEXT NULL,  
    fine_amount DECIMAL(10,2) NOT NULL,  
    notes TEXT NULL,  
    CONSTRAINT fk_viol_vehicle  
        FOREIGN KEY (vehicle_id) REFERENCES Vehicles(vehicle_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_viol_officer  
        FOREIGN KEY (officer_id) REFERENCES Officers(officer_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_viol_type  
        FOREIGN KEY (type_id) REFERENCES Violation_Types(type_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
--  
-- TABLE 7: Challans  
--
```

```
CREATE TABLE IF NOT EXISTS Challans (  
    challan_id INT AUTO_INCREMENT PRIMARY KEY,  
    challan_number VARCHAR(20) NOT NULL UNIQUE,  
    violation_id INT NOT NULL UNIQUE,  
    issue_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    due_date DATETIME NOT NULL,  
    total_amount DECIMAL(10,2) NOT NULL,  
    paid_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
    status ENUM('pending','partial','paid','cancelled') NOT NULL DEFAULT 'pending',  
    CONSTRAINT fk_challan_viol  
        FOREIGN KEY (violation_id) REFERENCES Violations(violation_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT chk_due_after_issue CHECK (due_date > issue_date)  
) ENGINE=InnoDB;
```

```
--  
-- TABLE 8: Payments  
--
```

```
CREATE TABLE IF NOT EXISTS Payments (  
    payment_id INT AUTO_INCREMENT PRIMARY KEY,  
    challan_id INT NOT NULL,  
    amount_paid DECIMAL(10,2) NOT NULL,  
    payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    payment_method ENUM('cash','card','online','bank_transfer') NOT NULL DEFAULT 'cash',  
    receipt_number VARCHAR(50) NULL,  
    processed_by INT NULL,  
    notes TEXT NULL,  
    CONSTRAINT fk_payment_challan  
        FOREIGN KEY (challan_id) REFERENCES Challans(challan_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_payment_officer  
        FOREIGN KEY (processed_by) REFERENCES Officers(officer_id)  
        ON DELETE SET NULL ON UPDATE CASCADE,  
    CONSTRAINT chk_amount_positive CHECK (amount_paid > 0)  
) ENGINE=InnoDB;
```

DDL Statements for Indices:

TABLE 1: USER

```
CREATE INDEX idx_users_role ON Users(role);
```

```
CREATE INDEX idx_users_active ON Users(is_active);
```

TABLE 2: OFFICER

```
CREATE INDEX idx_officer_badge ON Officers(badge_number);
```

TABLE 3: OWNER

```
CREATE INDEX idx_owner_cnic ON Owners(cnic);
```

```
CREATE INDEX idx_owner_name ON Owners(full_name);
```

TABLE 4: VEHICLE

```
CREATE INDEX idx_vehicle_reg ON Vehicles(registration_number);
```

```
CREATE INDEX idx_vehicle_owner ON Vehicles(owner_id);
```

TABLE 5: VIOLATION TYPES

```
CREATE INDEX idx_violation_types_name ON Violation_Types(type_name);  
CREATE INDEX idx_violation_types_severity ON Violation_Types(severity);  
CREATE INDEX idx_violation_types_active ON Violation_Types(is_active);  
CREATE INDEX idx_violation_types_fine ON Violation_Types(base_fine)
```

TABLE 6: VIOLATIONS

```
CREATE INDEX idx_viol_vehicle ON Violations(vehicle_id);  
CREATE INDEX idx_viol_officer ON Violations(officer_id);  
CREATE INDEX idx_viol_date ON Violations(violation_date);  
CREATE INDEX idx_viol_type ON Violations(type_id);
```

TABLE 7: CHALLANS

```
CREATE INDEX idx_challan_number ON Challans(challan_number);  
CREATE INDEX idx_challan_status ON Challans(status);  
CREATE INDEX idx_challan_due ON Challans(due_date);
```

TABLE 8: PAYMENT

```
CREATE INDEX idx_payment_challan ON Payments(challan_id);  
CREATE INDEX idx_payment_date ON Payments(payment_date);
```

LANGUAGES

FRONT_END:

HTML & Jinja2: HTML provides the structural skeleton for the web interface, building essential forms like the "Add Violation Type" page. Paired with Flask, Jinja2 acts as the templating engine to dynamically inject back-end data directly into these HTML pages. It handles dynamic URL routing and allows the creation of reusable structural layouts using template inheritance. This ensures a consistent and maintainable interface across the admin, officer, and payment dashboards.

CSS: CSS is utilized to style HTML elements, making the traffic management system visually appealing and easily navigable. Implemented through specific classes, it dictates the layout, colors, and typography of the entire web application. This styling ensures that data tables, input forms, and navigation menus are properly aligned. It also guarantees that the interface remains responsive and accessible across various devices used by administrators or officers.

JavaScript: JavaScript is responsible for client-side interactivity and dynamic behavior without requiring a full page reload. Within this project, it is utilized for instant form validation, such as ensuring valid numerical inputs for fines before submission. It also manages interactive UI elements on the dashboards, including pop-up confirmation modals and dropdown menus. Furthermore, it displays dynamic alerts when database actions, like recording a challan, are successfully completed.

BACK_END:

Python (Flask): Python serves as the core back-end engine, utilizing the Flask framework to seamlessly bridge the web front-end with the SQL database. It manages application routing by listening for web requests and securely processing incoming form data. Furthermore, Python executes the essential business logic of the system. This includes authenticating login sessions, processing payment data, and securely communicating with the database to save new records.

SQL: SQL is the fundamental language used to structure, manipulate, and query the relational database at the heart of the project. Data Definition Language (DDL) constructs the core tables that hold information on challans, vehicles, and officers. Additional scripts inject initial testing data and establish virtual views to optimize complex data retrieval. This structured approach makes it efficient for the system to generate rapid, comprehensive reports on traffic violations.

IMPLEMENTATION DESIGN

ADMIN:



Traffic FMIS

Metropolitan Traffic Authority

You have been logged out successfully.

Username

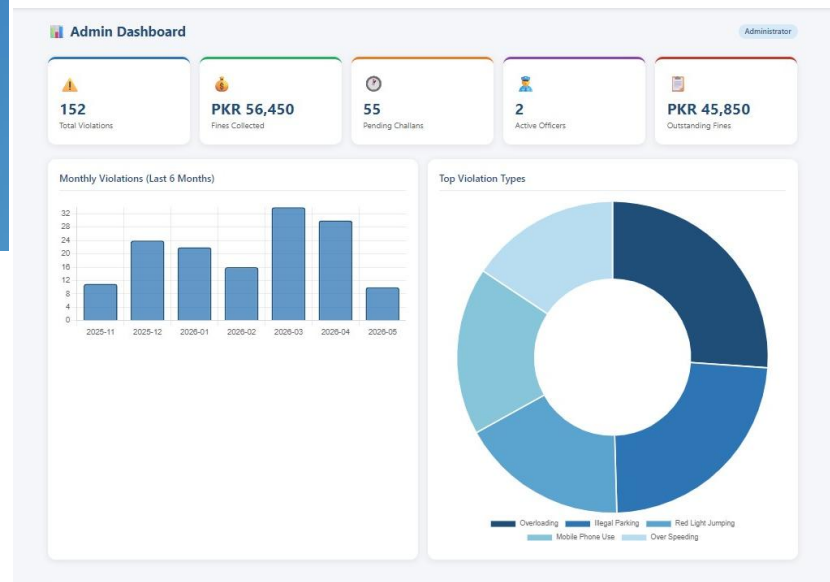
Enter your username

Password

Enter your password

Login

Traffic Violation & Fine Management System



+ Register New Officer

[← Back to Officers](#)

LOGIN CREDENTIALS

Username *

e.g. officer_ahmed

Email *

officer@traffic.gov

Password * (min 6 chars)

Minimum 6 characters

Account Role *

Traffic Officer

OFFICER DETAILS

Full Name *

Officer's full name

Badge Number *

e.g. TRF-001

Department

e.g. Traffic Enforcement

Precinct

e.g. District South

Phone Number

e.g. 0300-1234567

[Register Officer](#)[Cancel](#)

Traffic FMIS

[Dashboard](#)[All Challans](#)[Officers](#)[Violation Types](#)[Reports](#)[Add Officer](#)[Search](#)[admin](#)[Logout](#)

+ Register New Officer

[Back to Officers](#)

LOGIN CREDENTIALS

Username *

e.g. officer_ahmed

Email *

officer@traffic.gov

Password * (min 6 chars)

Minimum 6 characters

Account Role *

Traffic Officer

OFFICER DETAILS

Full Name *

Officer's full name

Badge Number *

e.g. TRF-001

Department

e.g. Traffic Enforcement

Precinct

e.g. District South

Phone Number

e.g. 0300-1234567

Register Officer

Cancel

Traffic FMIS

[Dashboard](#)[All Challans](#)[Officers](#)[Violation Types](#)[Reports](#)[Add Officer](#)[Search](#)[admin](#)[Logout](#)

Reports & Analytics

SQL Views Data

151
Total Challans

PKR 55,950
Total Revenue Collected

27
Repeat Offenders

55
Pending Challans

Monthly Revenue Summary

VIEW: vw_monthly_revenue

Amount Collected (PKR)

Number of Payments

PKR Collected

No. of Payments

2026-05

2026-04

2026-03

2026-02

2026-01

2025-12

2025-11

Month	Total Payments	Amount Collected (PKR)	Average Payment (PKR)
2026-05	7	3,600.00	514.29

The system administrator serves as the central authority with full operational and data oversight. Through a comprehensive dashboard, the admin monitors real-time analytics, including total revenue, pending challans, and repeat offender statistics. Administrators maintain direct control over personnel by viewing, editing, deleting, or toggling the login access of all registered traffic officers. Additionally, the admin configures core system rules by defining new traffic violation types and setting their corresponding base fines. They also possess the highest level of data management authority, allowing them to audit all issued challans, edit specific violation records, cancel invalid tickets, and generate detailed financial reports.

OFFICER:

Traffic FMIS

DashboardNew ViolationRegister VehicleSearchofficer_mairLogout

Officer Dashboard

Mair Nasir

70
My Violations

22
Pending Challans

PKR 28,850
Fines Collected

Record Violation

Register Vehicle

Search Challans

My Recent Violations

Date	Vehicle	Owner	Violation	Fine (PKR)	Challan No.	Status	Action
2026-05-19 16:48:00	KHI-7763-P	Hamid Gill	No Seatbelt	500	TRF-2026-000152	PAID	View
2026-05-17 02:08:00	KHI-9770-P	Babar Zaman	Mobile Phone Use	1,200	TRF-2026-000151	PENDING	View
2026-05-09 02:12:12	KHI-4291-C	Shahid Nawaz	Wrong Way Driving	1,500	KHI-2025-33128	PAID	View
2026-05-04 02:12:12	KHI-2343-W	Talha Yousuf	Illegal Parking	400	KHI-2025-95895	PARTIAL	View
2026-05-02 02:12:12	KHI-1515-P	Uzma Syed	Overloading	700	KHI-2025-36302	PENDING	View
2026-04-27 02:12:12	KHI-6878-W	Rabia Noor	Over Speeding	1,000	KHI-2025-36888	PENDING	View
2026-04-27 02:12:12	KHI-8258-D	Tariq Mehmood	Illegal Parking	400	KHI-2025-25250	PENDING	View
2026-04-25 02:12:12	KHI-9526-C	Anna Waheed	No Helmet	300	KHI-2025-42975	PAID	View
2026-04-20 02:12:12	KHI-9431-G	Babar Zaman	No Seatbelt	500	KHI-2025-86855	PARTIAL	View
2026-04-18 02:12:12	KHI-1515-P	Uzma Syed	Red Light Jumping	500	KHI-2025-65974	PENDING	View

Traffic FMIS

DashboardNew ViolationRegister VehicleSearchofficer_mairLogout

Register New Vehicle

Back

OWNER INFORMATION

If the CNIC already exists, owner record will be reused.

Owner CNIC * (13-15 digits)

Owner Full Name *

e.g. 4210112345678

Full legal name

Phone

Email

0300-1234567

owner@email.com

Address

Full residential address

VEHICLE INFORMATION

Registration Number * (e.g. ABC-123)

Vehicle Type *

E.G. LHR-5577

Car

Make (Brand)

Model

e.g. Toyota

e.g. Corolla

Year

Color

e.g. 2020

e.g. White

Register Vehicle

Cancel

Traffic FMIS

Dashboard

New Violation

Register Vehicle

Search

officer_mair

Logout

Record New Violation

Back

VEHICLE INFORMATION

Select Vehicle *

KHI-7763-P — Hyundai Tucson (Hamid Gill)

Vehicle not listed? Register it first →

VIOLATION DETAILS

Violation Type *

No Seatbelt — PKR 500 (MINOR)

Estimated Fine

Base Fine: Rs 500

* Final fine may be higher for repeat offenders

Date & Time of Violation *

05/19/2026 04:48 PM

Location *

University Road near Hassan Square

Additional Notes

Any additional details about the violation...

Record Violation & Generate Challan

Cancel

Traffic officers operate as the primary field agents within the system, utilizing a personalized dashboard to track their active enforcement metrics. They are authorized to formally register new vehicles and their owners into the central database. Their main operational duty is to record live traffic violations, prompting the system to calculate fines, apply repeat offender penalties, and generate formal challan numbers. Crucially, the system enforces strict data access controls; an officer is exclusively permitted to view, monitor the payment status of, and download PDF records for only those challans they have personally generated, completely restricting access to tickets issued by other staff.

AUTHENTICATION AND AUTHORIZATION:

The authentication module serves as the central security gateway for the entire system, managing access control and user identity. It securely processes login requests by verifying credentials, establishing active user sessions, and automatically routing personnel to their appropriate role-based dashboards. To protect sensitive data, it utilizes strict authorization rules that prevent unauthorized access to restricted system interfaces. Furthermore, this module handles the safe logout process and empowers administrators to register new system users, ensuring that all user passwords are securely encrypted before being stored in the database.

PAYMENT:

Traffic FMIS

DashboardNew ViolationRegister VehicleSearchofficer_mairLogout

Record Payment

Back to Challan

Challan No:TRF-2026-000152Vehicle:KHI-7763-POwner:Hamid GillViolation:No SeatbeltDue Date:2026-06-03 16:48:43

Original Fine:PKR 500.00

Already Paid:- PKR 0.00

Outstanding Balance:PKR 500.00

ENTER PAYMENT

Payment Amount (PKR) *Payment Method *

500.0Cash

Maximum: PKR 500.00

Confirm PaymentCancel

Traffic FMIS

[Dashboard](#)[New Violation](#)[Register Vehicle](#)[Search](#)[officer_mair](#)[Logout](#)

Challan Details

[Download PDF](#)[Back](#)

Challan Status: PAID

Challan Information

Challan Number

TRF-2026-000152

Issue Date

2026-05-19 16:48:42

Due Date

2026-06-03 16:48:43

Vehicle & Owner

Registration

KHI-7763-P

Vehicle

2013 Hyundai Tucson (Silver)

Owner

Hamid Gill

CHNIC

4230643309136

Phone

03326197160

Violation Details

Violation Type

No Seatbelt

MINOR

Date & Time

2026-05-19 16:48:00

Location

University Road near Hassan Square

Description

—

Issuing Officer

Mair Nasir (TRF-002)

Fine Summary

Original Fine

PKR 500.00

Late Penalty

PKR 0.00

Total Amount

PKR 500.00

Amount Paid

PKR 500.00

Balance Due

PKR 0.00

Payment History

Receipt No.	Date	Amount	Method	Processed By
RCP-51F8D313	2026-05-19 17:00:48	PKR 500.00	ONLINE	Mair Nasir

The payment processing module handles the secure collection and recording of fine payments for issued challans. Authorized personnel can retrieve a specific ticket to view its outstanding balance, which dynamically includes any applicable late penalties. During a transaction, the system automatically validates the entered payment to ensure it is a valid amount and does not exceed the total due. Upon a successful transaction, it generates a unique alphanumeric receipt number and logs the payment method. The software then immediately updates the database, adjusting the total collected amount and automatically shifting the challan's status to either 'partial' or fully 'paid'.

SEARCH:

Traffic FMIS

DashboardNew ViolationRegister VehicleSearchofficer_mainLogout

Search Challans & Violations

Vehicle Registration No.
e.g. LHR-5577

Owner CNIC
e.g. 4210112345678

Challan Number
e.g. TRF-2024-000001

Date From
mm/dd/yyyy

Date To
mm/dd/yyyy

Status
All Statuses

Search

Clear

Search Challans & Violations

Vehicle Registration No.
KHI-7763-P

Owner CNIC
e.g. 4210112345678

Challan Number
e.g. TRF-2024-000001

Date From
mm/dd/yyyy

Date To
mm/dd/yyyy

Status
All Statuses

Search

Clear

Search Results2 records

Challan No.	Vehicle	Owner	CNIC	Violation	Date	Location	Total (PKR)	Status	Action
TRF-2026-000152	KHI-7763-P	Hamid Gill	4230643309136	No Seatbelt	2026-05-19	University Road near Hassan Square	500	PAID	View
KHI-2025-83289	KHI-7763-P	Hamid Gill	4230643309136	Overloading	2025-11-23	Tipu Sultan Road near...	700	PENDING	View

The universal search module provides a comprehensive querying tool for all authorized personnel to quickly locate specific traffic records. Users can filter the database using a variety of precise criteria, including vehicle registration numbers, owner CNICs, unique challan numbers, or specific date ranges. Additionally, records can be narrowed down by the issuing officer's name or the current payment status of the ticket. The system dynamically cross-references multiple data tables to retrieve and display accurate, up-to-date challan details instantly.

FUTURE EXPANSION

Public Citizen Portal: The system can be expanded to include a secure public-facing portal where vehicle owners can log in using their CNIC or vehicle registration number. This would allow citizens to independently check their violation history, view pending challans, and download PDF copies of their tickets without needing to visit a traffic office.

Online Payment Gateway Integration: Currently, payments are processed manually by officers. A critical future upgrade involves integrating third-party digital payment APIs, enabling citizens to pay their fines directly through bank cards or local mobile wallets like JazzCash and EasyPaisa. This would automate the status updates from "pending" to "paid" in real-time.

Automated SMS and Email Alerts: To improve compliance and fine recovery rates, an automated notification module can be developed. By integrating SMS or email services, the system could automatically alert vehicle owners the moment a challan is generated, and send automated reminders before the due date or warning alerts when a penalty is applied.

Dedicated Mobile Application for Field Officers: While the current web-based interface is fully functional, developing a dedicated mobile application for on-ground traffic officers would significantly enhance field efficiency. A mobile app could utilize the smartphone's camera for Automated Number Plate Recognition (ANPR), instantly scanning license plates to auto-fill registration details in the violation form.

Course Code: CS 222, Course Title: Database Management Systems

Evaluation Rubric

CLO: CLO-2, Taxonomy Level: C3

CRITERIA AND SCALES			
Criterion 1: Database Creation (Tables and Structure)			
0	1	2	3
Tables not created or incorrect structure	Basic tables created but with major errors	Tables created correctly with minor errors	All tables correctly created with proper structure
Criterion 2: Keys and Constraints Implementation			
0	1	2	3
No primary keys or constraints defined	Primary keys defined but foreign keys missing or incorrect	Primary and foreign keys defined correctly	All keys and constraints correctly implemented with integrity enforced
Criterion 3: Data Manipulation Operations (Insert, Update, Delete)			
0	1	2	3
Operations not implemented	Only insert operation implemented	Insert, update, delete mostly implemented	All insert, update, delete operations correctly implemented
Criterion 4: Query Implementation			
0	1	2	3
Queries not implemented	Basic select queries only	Select queries with joins implemented	Complex queries with joins, views, filtering, and sorting implemented
Criterion 5: Front-End and Database Connectivity			
0	1	2	3
No front-end or no connectivity	Front-end exists but connection issues present	Front-end connected with minor issues	Fully functional front-end connected with database
Criterion 6: Reports Generation			
0	1	2	3
No reports generated	Basic report generated	Multiple useful reports generated	Professional and meaningful reports generated
Criterion 7: Answer to questions			
0	1	2	
Unable to answer questions	Answers basic questions only	Shows excellent understanding of the project	